



INF1040 – PROJETO DE PROGRAMAÇÃO MODULAR (06/05/2026)

TRABALHO 02

- **Nome:** Alexandre Scofield **Matrícula:** 2311672
- **Nome:** Filipe Izidoro Reis **Matrícula:** 2410329
- **Nome:** Carina Cavalcante **Matrícula:** 2511668
- **Nome:** João Pedro Theodoro **Matrícula:** 2510968
- **Turma:** 3WB

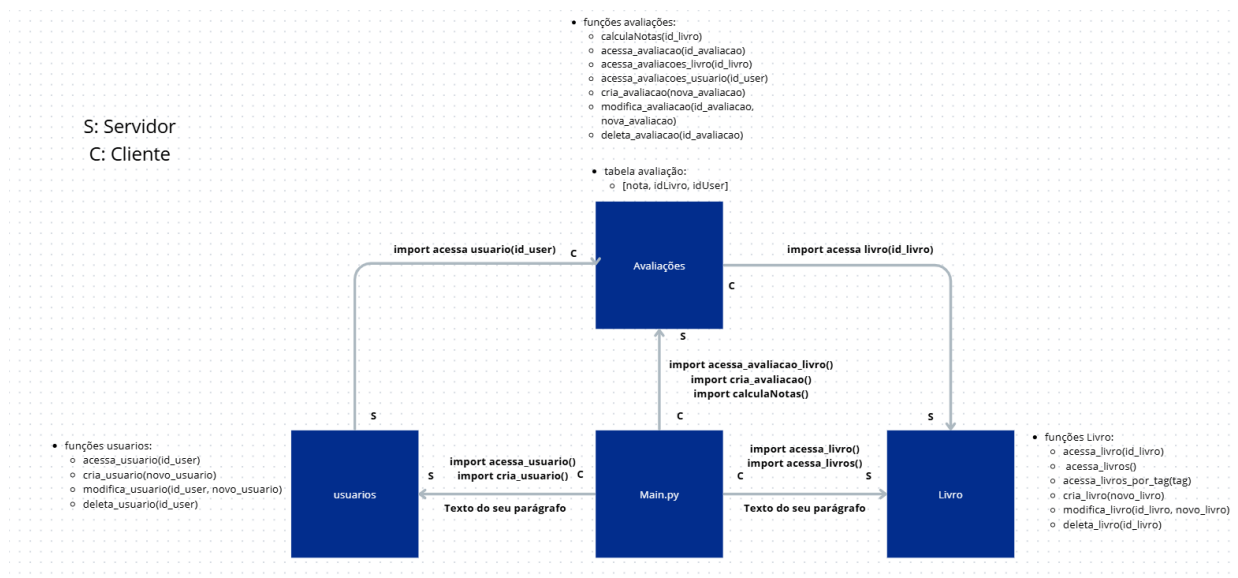
Rio de Janeiro

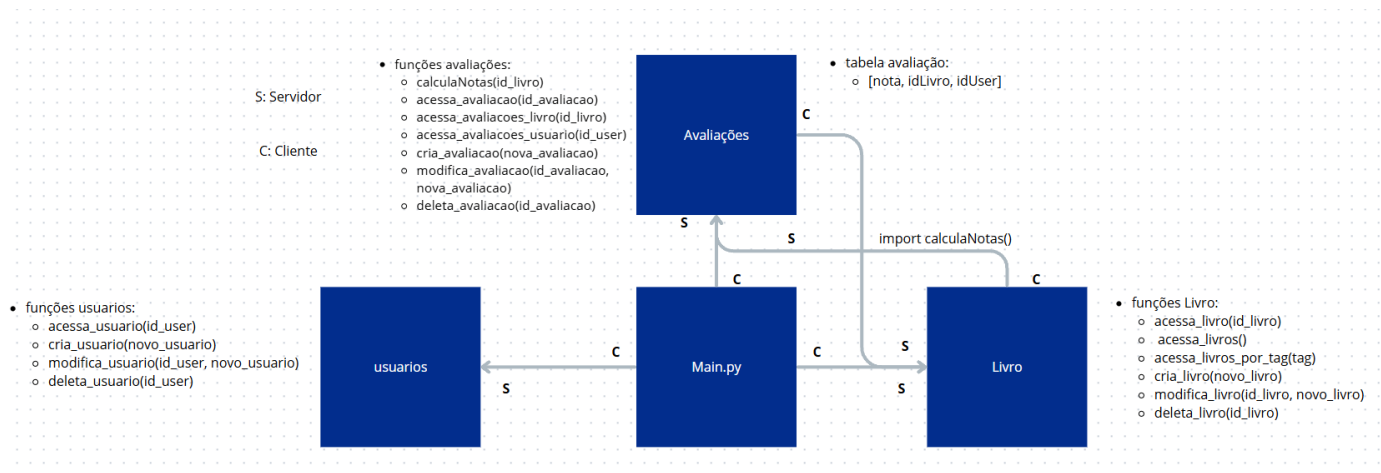
Novembro de 2025

OBJETIVO:

O objetivo do projeto é desenvolver uma aplicação modular para cadastro, consulta e avaliação de livros, permitindo que usuários registrem seus perfis, acessem livros cadastrados e atribuam notas a eles. A aplicação será organizada em módulos com responsabilidades bem definidas, separando o controle de usuários, livros e avaliações, de forma que cada parte do sistema trate um conceito específico. Com isso, o projeto busca facilitar a manutenção, a evolução e a reutilização do código, seguindo os princípios de programação modular vistos na disciplina.

DIAGRAMA DE MODULARIZAÇÃO:





REQUISITOS:

1. Requisitos Funcionais

- A aplicação armazenará os dados dos livros cadastrados.
- A aplicação armazenará os dados dos usuários cadastrados.
- A aplicação armazenará os dados das avaliações feitas pelos usuários.
- A aplicação permitirá que um usuário registre seu perfil no sistema.
- A aplicação informará se o registro do perfil do usuário foi concluído com sucesso ou não.
- A aplicação permitirá que o usuário consulte os livros cadastrados.
- A aplicação permitirá que o usuário acesse os dados de um livro cadastrado.
- A aplicação permitirá que o usuário selecione um livro para avaliação.
- A aplicação permitirá que o usuário cadastre uma avaliação para um livro.
- Ao cadastrar uma avaliação, a aplicação armazenará a nota (0 a 5), a identificação do livro avaliado e a identificação do usuário avaliador.
- A aplicação permitirá consultar as avaliações associadas a um livro.

- A aplicação permitirá calcular ou obter a nota de um livro a partir de suas avaliações.
- O módulo Livro armazenará, para cada livro, sua identificação, seu nome, seu autor e suas tags.
- O módulo Avaliações armazenará, para cada avaliação, a nota, a identificação do livro e a identificação do usuário.
- A aplicação permitirá que a busca de notas de um livro seja feita a partir da identificação do livro.
- A aplicação permitirá que o usuário avalie um livro sem precisar digitar manualmente sua identificação, usando uma opção da interface vinculada ao ID do livro.
- A aplicação permitirá que o módulo principal (Main.py) acesse as funcionalidades de usuários, livros e avaliações.
- A aplicação permitirá que o módulo Livro utilize a funcionalidade de cálculo de notas fornecida pelo módulo Avaliações.

2. Requisitos Não Funcionais

- A aplicação será organizada em módulos com responsabilidades separadas: usuários, livros, avaliações e programa principal. Isso segue a ideia de programação modular do curso, em que o programa é dividido em módulos com objetivos bem definidos.
- Cada módulo deverá tratar um único conceito principal, para manter boa coesão: o módulo usuários tratará usuários, o módulo Livro tratará livros e o módulo Avaliações tratará avaliações.
- A comunicação entre os módulos deverá ocorrer por interfaces bem definidas, usando dados como idLivro, idUser e retornos de sucesso ou falha. O material define interface como mecanismo de troca de dados, comandos e eventos entre elementos de software.

- Os dados internos de cada módulo deverão ser protegidos contra uso indevido por outros módulos, respeitando o princípio de encapsulamento apresentado na disciplina.
- As funções de acesso deverão validar os dados recebidos antes de realizar operações, principalmente identificações de livros, identificações de usuários e notas.
- As consultas de livros e avaliações deverão retornar em tempo adequado (3 segundos) para uso interativo.
- Caso exista autenticação no projeto, a segurança da conta deverá ser feita por login e senha específicos para cada usuário.
- A aplicação deverá ser implementada em Python, com os módulos separados conforme a arquitetura indicada no diagrama.
- O sistema deve restringir o usuário para avaliar o mesmo livro apenas uma vez.
- O acesso a funcionalidades só é permitido após autenticação.
- Só poderá ser feito cadastro com e-mails que ainda não foram registrados no sistema.
- A aplicação deverá evitar ambiguidade na interface com o usuário, apresentando claramente qual livro está sendo consultado ou avaliado.
- A aplicação deve impedir que uma avaliação seja registrada sem um livro e um usuário associados.

Dados encapsulados:

1. Módulo usuários

- E-mail
- Senha

2. Módulo Avaliações

- Cálculo geral das avaliações
- ID avaliação
- ID livro

- E-mail

3. Módulo Livros

- ID do livro
- Tags
- Autor

FUNÇÕES DE ACESSO:

1. Módulo usuários.py

```
def acessa_usuario(id_user):
```

```
    """Retorna os dados de um usuário a partir do seu ID."""
```

```
    pass
```

```
def cria_usuario(novo_usuario):
```

```
    """Cadastra um novo usuário no sistema."""
```

```
    pass
```

```
def modifica_usuario(id_user, novo_usuario):  
  
    """Modifica os dados de um usuário já cadastrado."""  
  
    pass
```

```
def deleta_usuario(id_user):  
  
    """Remove um usuário cadastrado do sistema."""  
  
    pass
```

2. Módulo livro.py

```
def acessa_livro(id_livro):  
  
    """Retorna os dados de um livro a partir do seu ID."""  
  
    pass
```

```
def acessa_livros():  
  
    """Retorna todos os livros cadastrados."""  
  
    pass
```

```
def acessa_livros_por_tag(tag):  
    """Retorna os livros associados a uma determinada tag."""  
  
    pass
```

```
def cria_livro(novo_livro):  
    """Cadastra um novo livro no sistema."""  
  
    pass
```

```
def modifica_livro(id_livro, novo_livro):  
    """Modifica os dados de um livro já cadastrado."""  
  
    pass
```

```
def deleta_livro(id_livro):  
    """Remove um livro cadastrado do sistema."""  
  
    pass
```


3. Módulo avaliações.py

```
def acessa_avaliacao(id_avaliacao):  
    """Retorna uma avaliação a partir do seu ID."""  
  
    pass  
  
def acessa_avaliacoes_livro(id_livro):  
    """Retorna todas as avaliações associadas a um livro."""  
  
    pass  
  
def acessa_avaliacoes_usuario(id_user):  
    """Retorna todas as avaliações feitas por um usuário."""  
  
    pass  
  
def cria_avaliacao(nova_avaliacao):  
    """Cadastra uma nova avaliação para um livro."""  
  
    pass
```

```
def modifica_avaliacao(id_avaliacao, nova_avaliacao):
```

```
    """Modifica uma avaliação já cadastrada."""
```

```
    pass
```

```
def deleta_avaliacao(id_avaliacao):
```

```
    """Remove uma avaliação cadastrada."""
```

```
    pass
```

```
def calcula_notas(id_livro):
```

```
    """Calcula a nota de um livro a partir das avaliações  
    cadastradas."""
```

```
    Pass
```

TESTES DAS FUNÇÕES:

Casos de Teste Previstos

Convenção dos Códigos de Retorno

Os módulos da aplicação utilizarão códigos inteiros padronizados para indicar o resultado das operações executadas:

- 0: operação realizada com sucesso.
- 1: registro não encontrado.
- 2: dados inválidos.
- 3: identificador já cadastrado.
- 4: livro inexistente.
- 5: usuário inexistente.
- 6: nota inválida.
- 7: operação não permitida.

Módulo usuarios.py

Função: `acessa_usuario(id_user)`

Caso 1 – Usuário encontrado

- Pré-condição: o usuário existe.
- Execução: chamar `acessa_usuario(id_user)`.
- Retorno esperado: 0.

Caso 2 – Usuário não encontrado

- Pré-condição: o usuário não existe.
- Execução: chamar `acessa_usuario(id_user)` com identificador inexistente.
- Retorno esperado: 1.

Função: `cria_usuario(novo_usuario)`

Caso 1 – Usuário criado com sucesso

- Pré-condição: o usuário não está cadastrado e os dados são válidos.
- Execução: chamar `cria_usuario(novo_usuario)`.
- Retorno esperado: 0.

Caso 2 – Identificador já cadastrado

- Pré-condição: já existe um usuário com o mesmo identificador.
- Execução: chamar `cria_usuario(novo_usuario)` com identificador já existente.
- Retorno esperado: 3.

Caso 3 – Dados inválidos

- Pré-condição: algum campo obrigatório é inválido.
- Execução: chamar `cria_usuario(novo_usuario)` com dados inválidos.
- Retorno esperado: 2.

Função: `modifica_usuario(id_user, novo_usuario)`

Caso 1 – Usuário modificado com sucesso

- Pré-condição: o usuário existe e os novos dados são válidos.
- Execução: chamar `modifica_usuario(id_user, novo_usuario)`.
- Retorno esperado: 0.

Caso 2 – Usuário não encontrado

- Pré-condição: o usuário não existe.
- Execução: chamar `modifica_usuario(id_user, novo_usuario)` com identificador inexistente.
- Retorno esperado: 1.

Caso 3 – Dados inválidos

- Pré-condição: os novos dados são inválidos.
- Execução: chamar `modifica_usuario(id_user, novo_usuario)`.
- Retorno esperado: 2.

Função: `deleta_usuario(id_user)`

Caso 1 – Usuário removido com sucesso

- Pré-condição: o usuário existe.
- Execução: chamar `deleta_usuario(id_user)`.
- Retorno esperado: 0.

Caso 2 – Usuário não encontrado

- Pré-condição: o usuário não existe.
- Execução: chamar `deleta_usuario(id_user)`.
- Retorno esperado: 1.

Módulo livro.py

Função: `acessa_livro(id_livro)`

Caso 1 – Livro encontrado

- Pré-condição: o livro existe.
- Execução: chamar `acessa_livro(id_livro)`.
- Retorno esperado: 0.

Caso 2 – Livro não encontrado

- Pré-condição: o livro não existe.
- Execução: chamar `acessa_livro(id_livro)`.
- Retorno esperado: 1.

Função: `acessa_livros()`

Caso 1 – Existem livros cadastrados

- Pré-condição: existe pelo menos um livro cadastrado.
- Execução: chamar `acessa_livros()`.
- Retorno esperado: 0.

Caso 2 – Nenhum livro cadastrado

- Pré-condição: não existem livros cadastrados.
- Execução: chamar `acessa_livros()`.
- Retorno esperado: 1.

Função: `acessa_livros_por_tag(tag)`

Caso 1 – Livros encontrados

- Pré-condição: existem livros associados à tag informada.
- Execução: chamar `acessa_livros_por_tag(tag)`.
- Retorno esperado: 0.

Caso 2 – Nenhum livro encontrado

- Pré-condição: não existem livros associados à tag informada.
- Execução: chamar `acessa_livros_por_tag(tag)`.
- Retorno esperado: 1.

Caso 3 – Tag inválida

- Pré-condição: a tag é inválida.
- Execução: chamar `acessa_livros_por_tag(tag)`.
- Retorno esperado: 2.

Função: `cria_livro(novo_livro)`

Caso 1 – Livro criado com sucesso

- Pré-condição: o livro não está cadastrado e os dados são válidos.
- Execução: chamar `cria_livro(novo_livro)`.
- Retorno esperado: 0.

Caso 2 – Identificador já cadastrado

- Pré-condição: já existe livro com o mesmo identificador.
- Execução: chamar `cria_livro(novo_livro)`.
- Retorno esperado: 3.

Caso 3 – Dados inválidos

- Pré-condição: algum dado obrigatório é inválido.
- Execução: chamar `cria_livro(novo_livro)`.
- Retorno esperado: 2.

Função: `modifica_livro(id_livro, novo_livro)`

Caso 1 – Livro modificado com sucesso

- Pré-condição: o livro existe e os novos dados são válidos.
- Execução: chamar `modifica_livro(id_livro, novo_livro)`.
- Retorno esperado: 0.

Caso 2 – Livro não encontrado

- Pré-condição: o livro não existe.
- Execução: chamar `modifica_livro(id_livro, novo_livro)`.
- Retorno esperado: 1.

Caso 3 – Dados inválidos

- Pré-condição: os novos dados são inválidos.
- Execução: chamar `modifica_livro(id_livro, novo_livro)`.
- Retorno esperado: 2.

Função: `deleta_livro(id_livro)`

Caso 1 – Livro removido com sucesso

- Pré-condição: o livro existe.
- Execução: chamar `deleta_livro(id_livro)`.
- Retorno esperado: 0.

Caso 2 – Livro não encontrado

- Pré-condição: o livro não existe.

- Execução: chamar `deleta_livro(id_livro)`.
- Retorno esperado: 1.

Módulo `avaliacoes.py`

Função: `acessa_avaliacao(id_avaliacao)`

Caso 1 – Avaliação encontrada

- Pré-condição: a avaliação existe.
- Execução: chamar `acessa_avaliacao(id_avaliacao)`.
- Retorno esperado: 0.

Caso 2 – Avaliação não encontrada

- Pré-condição: a avaliação não existe.
- Execução: chamar `acessa_avaliacao(id_avaliacao)`.
- Retorno esperado: 1.

Função: `acessa_avaliacoes_livro(id_livro)`

Caso 1 – Avaliações encontradas

- Pré-condição: o livro existe e possui avaliações.
- Execução: chamar `acessa_avaliacoes_livro(id_livro)`.
- Retorno esperado: 0.

Caso 2 – Livro sem avaliações

- Pré-condição: o livro existe, mas não possui avaliações.
- Execução: chamar `acessa_avaliacoes_livro(id_livro)`.
- Retorno esperado: 1.

Caso 3 – Livro inexistente

- Pré-condição: o livro não existe.

- Execução: chamar `acessa_avaliacoes_livro(id_livro)`.
- Retorno esperado: 4.

Função: `acessa_avaliacoes_usuario(id_user)`

Caso 1 – Avaliações encontradas

- Pré-condição: o usuário existe e possui avaliações.
- Execução: chamar `acessa_avaliacoes_usuario(id_user)`.
- Retorno esperado: 0.

Caso 2 – Usuário sem avaliações

- Pré-condição: o usuário existe, mas não possui avaliações.
- Execução: chamar `acessa_avaliacoes_usuario(id_user)`.
- Retorno esperado: 1.

Caso 3 – Usuário inexistente

- Pré-condição: o usuário não existe.
- Execução: chamar `acessa_avaliacoes_usuario(id_user)`.
- Retorno esperado: 5.

Função: `cria_avaliacao(nova_avaliacao)`

Caso 1 – Avaliação criada com sucesso

- Pré-condição: livro e usuário existem e a nota é válida.
- Execução: chamar `cria_avaliacao(nova_avaliacao)`.
- Retorno esperado: 0.

Caso 2 – Livro inexistente

- Pré-condição: o livro informado não existe.
- Execução: chamar `cria_avaliacao(nova_avaliacao)`.
- Retorno esperado: 4.

Caso 3 – Usuário inexistente

- Pré-condição: o usuário informado não existe.
- Execução: chamar `cria_avaliacao(nova_avaliacao)`.
- Retorno esperado: 5.

Caso 4 – Nota inválida

- Pré-condição: a nota está fora do intervalo permitido.
- Execução: chamar `cria_avaliacao(nova_avaliacao)`.
- Retorno esperado: 6.

Função: `modifica_avaliacao(id_avaliacao, nova_avaliacao)`

Caso 1 – Avaliação modificada com sucesso

- Pré-condição: a avaliação existe e os novos dados são válidos.
- Execução: chamar `modifica_avaliacao(id_avaliacao, nova_avaliacao)`.
- Retorno esperado: 0.

Caso 2 – Avaliação não encontrada

- Pré-condição: a avaliação não existe.
- Execução: chamar `modifica_avaliacao(id_avaliacao, nova_avaliacao)`.
- Retorno esperado: 1.

Caso 3 – Dados inválidos

- Pré-condição: os novos dados são inválidos.
- Execução: chamar `modifica_avaliacao(id_avaliacao, nova_avaliacao)`.
- Retorno esperado: 2.

Função: `deleta_avaliacao(id_avaliacao)`

Caso 1 – Avaliação removida com sucesso

- Pré-condição: a avaliação existe.
- Execução: chamar `deleta_avaliacao(id_avaliacao)`.
- Retorno esperado: 0.

Caso 2 – Avaliação não encontrada

- Pré-condição: a avaliação não existe.
- Execução: chamar `deleta_avaliacao(id_avaliacao)`.
- Retorno esperado: 1.

Função: `calcula_notas(id_livro)`

Caso 1 – Nota calculada com sucesso

- Pré-condição: o livro existe e possui avaliações.
- Execução: chamar `calcula_notas(id_livro)`.
- Retorno esperado: 0.

Caso 2 – Livro sem avaliações

- Pré-condição: o livro existe, mas não possui avaliações.
- Execução: chamar `calcula_notas(id_livro)`.
- Retorno esperado: 1.

Caso 3 – Livro inexistente

- Pré-condição: o livro não existe.
- Execução: chamar `calcula_notas(id_livro)`.
- Retorno esperado: 4.

Responsabilidades dos desenvolvedores:

Módulo	Responsável
Usuarios.py	Alexandre
Livro.py	Filipe
Avalicoes.py	Carina
Main	João